

V8のArray.prototype.flatを 最大約20倍速くするまでと、 巨大OSSへの大規模コミットの道のり

西 悠太 / 株式会社ダイニー

自己紹介

西 悠太 (Nishi Yuta)

21歳(重要、important)

V8 Contributor

Platform Engineer at Dinii Inc.

TSKaigi Staff

TypeScriptが好きです

V8にパッチを送るのも好きです

あと野球も好きです

@riya-amemiya



そもそも何でこのパッチを出したのか

ある日、Xのタイムラインに「JSCのArray.prototype.flatが速くなる」という投稿が流れてきました

実装を読むと同じ発想がV8にも適用できそうに見えました

約1ヶ月後、私のパッチはV8 14.7にマージされました

**Chrome、Node.js、Deno、V8を積む
あらゆるランタイムでflatが最大約20倍速くなります**

最適化メインで話します

- なぜ従来のflatは遅かったのか
- メモリ割り当てを1回に減らす2パス方式
- holeを詰めるflat特有の仕様との格闘
- V8独自言語Torqueでの実装

上記について順を追って解説します

v8-devでの提案からGerritレビュー、マージまでの流れも紹介します

なぜ従来のflatは
遅かったのか

flatの仕様を理解する

```
> [[1, 2], [3, 4]].flat()  
[ 1, 2, 3, 4 ]  
  
> [[[1]], [[2]]].flat(2)  
[ 1, 2 ]
```

- `depth` 段までフラットにした新しい配列を返す
- `depth` を省略すると1

holeの扱い

JavaScriptの配列では、要素が存在しないインデックスのことをholeと呼びます

```
> [1, , 3]  
[ 1, <1 empty item>, 3 ]
```

holeは `undefined` と違い、プロパティ自体がない

従来のflatの実装

1. 長さ0の配列を作る
2. ソース配列を先頭から1個ずつ見る
3. 要素が配列なら、再帰呼び出しでその中を見る
4. 配列でなければ、結果配列の末尾に1個追加する

どこが遅かったのか

- 容量が足りなくなるたびに確保し直してコピー
- 汎用のプロパティ書き込みで1個ずつ追加
- サブ配列ごとに再帰呼び出し

最適化前

長さ0の配列から始める

入力は要素10個のサブ配列6個



結果配列の容量は 0

確保 0 回 書き込み 0 回 コピーし直し 0 個 再帰呼び出し 0 回

新しい容量 = 必要な長さ + 必要な長さの半分 + 16

ElementsKindとは

V8は配列の中身を

「整数だけ」「浮動小数点を含む」「Holeがある」といった
ElementsKindと呼ばれる内部型で追跡しており、

この型情報を使うと

「数値しか入っていない配列にはサブ配列が存在し得ない」ことが自明になり、
走査そのものを省略できます

主要なElementsKind

	Packed (holeなし)	Holey (holeあり)
小さな整数 (Smi) だけ	PACKED_SMI_ELEMENTS	HOLEY_SMI_ELEMENTS
浮動小数点数を含む	PACKED_DOUBLE_ELEMENTS	HOLEY_DOUBLE_ELEMENTS
文字列やオブジェクトを含む	PACKED_ELEMENTS	HOLEY_ELEMENTS

遷移は原則として一方向

HOLEYになったら、holeを埋めてもPACKEDには戻らない

`PACKED_SMI_ELEMENTS` と `PACKED_DOUBLE_ELEMENTS` には
数値しか入らない

- サブ配列は入らない
- Proxyも入らない
- holeもない

この保証を使って最適化する

メモリ割り当てを
1回に減らす2パス方式

最適化の戦略

JSCと同じ2パス方式

1. 第1パスで結果の長さと ElementsKindを求める
2. その長さで1回だけ確保
3. 第2パスで要素を書き込む

参考にした Sosuke Suzuki さんのPR

<https://github.com/WebKit/WebKit/pull/56035>

結果配列を何回確保するか

要素10個のサブ配列6個の場合

最適化前



容量17で確保



容量43で確保し直し、17個をコピー



容量82で確保し直し、43個をコピー

2パス方式



容量60で1回だけ確保

2パス方式

第1パスで長さを数える

入力是要素10個のサブ配列6個



第1パスで数えた長さは 0

確保 0回 書き込み 0回 確認 0回

2パス方式で変わること

	最適化前	2パス方式
メモリ割り当て	$O(\log n)$ 回	1回
要素の書き込み	汎用のプロパティ書き込み	確保した領域に直接書く
ElementsKindの遷移	起こりうる	起こらない

数値配列のショートカット

- ソースが `PACKED_SMI_ELEMENTS` か `PACKED_DOUBLE_ELEMENTS` なら、中身は数値だけでholeもない
- だから要素を見ずに `.length` をそのまま結果の長さにする

```
// src/builtins/array-flat.tq
if (sourceKind == ElementsKind::PACKED_SMI_ELEMENTS ||
    sourceKind == ElementsKind::PACKED_DOUBLE_ELEMENTS) {
  return FlattenedLengthResult{length: sourceLength, targetKind: sourceKind};
}
```

`[1, 2, 3, ..., 1024].flat()` の長さ計算は $O(1)$

サブ配列に対するショートカット

数値だけのサブ配列は、中を読まずに `.length` を足す

```
// src/builtins/array-flat.tq
if (rawKind == ElementsKind::PACKED_SMI_ELEMENTS) {
  seenSmi = true;
  const subLen: Smi =
    Cast<Smi>(elementArray.length) otherwise goto Bailout;
  targetLength =
    math::TrySmiAdd(targetLength, subLen) otherwise goto Bailout;
  index++;
  continue;
}
```

```
[[1,2,3], [4,5,6], [7,8,9]].flat()
```

見るのは外側の3要素だけ

外側の配列は `PACKED_ELEMENTS`



結果の長さは 9

ElementsKindの追跡

長さを数えながら、結果に入る値の型を3つのフラグで追う

```
// src/builtins/array-flat.tq
if (!IsNumber(element)) {
  seenObject = true;
} else if (!TaggedIsSmi(element)) {
  seenDouble = true;
} else {
  seenSmi = true;
}
```

走査が終わったら、フラグから結果のElementsKindを決める

```
// src/builtins/array-flat.tq
if (seenObject) {
  targetKind = ElementsKind::PACKED_ELEMENTS;
} else if (seenDouble) {
  targetKind = ElementsKind::PACKED_DOUBLE_ELEMENTS;
} else {
  targetKind = ElementsKind::PACKED_SMI_ELEMENTS;
}
```

`[[1], [2], [3]].flat()` の結果は `PACKED_SMI_ELEMENTS`

型が混ざった配列

`[[1, 2], [3.5], ["a", "b"]].flat()` の場合

最適化前



2パス方式

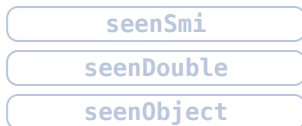


型が混ざった配列

入力は [[1, 2], [3.5], ["a", "b"]]

最適化前

2パス方式



最適化前は1個ずつ足していく

確保 0 回 コピーし直し 0 個

確保 0 回 コピーし直し 0 個

SmiとDoubleが混ざると PACKED_DOUBLE_ELEMENTS

```
// test/mjsunit/array-flat-elements-kind.js
assertSmiElementsKind([[1],[1]].flat());
assertDoubleElementsKind([[1],[1.1]].flat());
assertObjectElementsKind([["hello"]].flat());
```

SmiはDoubleで表せるので、
PACKED_ELEMENTS まで広げない

flatのあとの処理も速くなる

- JITはElementsKindに合わせて特殊化したコードを作る
- 結果が `PACKED_SMI_ELEMENTS` なら、後のループで型チェックを省ける

第2パスはまだ1個ずつコピーしていた

- [1, 2, 3] はメモリ上では整数が3つ並んでいるだけ
- まとめて移せば済むのに、1個ずつ読み書きしていた

要素1個ごとに4つ確認する

- 配列の構造が変わっていないか (Recheck)
- holeでないか
- Proxyでないか
- 書き込み先からあふれないか

2パス方式があるからできること

- 第1パスでコピー先の長さと型はもう決まっている
- 配列を作り直さなくていい
- 要素ごとに型をそろえ直さなくていい

数値のサブ配列はまとめてコピー

サブ配列 [1, 2, 3] のメモリ



↓ memcpy で3個まとめてコピー

結果配列のメモリ



バルクコピー

第1パスで長さを数える

入力是要素10個のサブ配列6個



第1パスで数えた長さは 0

確保 0回 memcpy 0回 確認 0回

TorqueCopyElements は最終的に libc の memcpy を呼ぶ

```
// src/builtins/array-flat.tq
if (subArray.map.elements_kind == ElementsKind::PACKED_SMI_ELEMENTS) {
  const srcElements: FixedArray = Cast<FixedArray>(subArray.elements)
    otherwise goto Bailout;
  const srcLen: Smi = Cast<Smi>(subArray.length) otherwise goto Bailout;
  const newIdx: Smi = math::TrySmiAdd(targetIndex, srcLen)
    otherwise goto Bailout;
  if (Convert<intptr>(newIdx) > vector.fixedArray.length_intptr) {
    goto Bailout;
  }
  TorqueCopyElements(
    vector.fixedArray, SmiUntag(targetIndex), srcElements, 0,
    SmiUntag(srcLen));
  targetIndex = newIdx;
  index++;
  continue;
}
```

PACKED_DOUBLE_ELEMENTS も同じように FixedDoubleArray をコピー

まとめてコピーできるのは数値配列だけ

- 対象は `PACKED_SMI_ELEMENTS` と `PACKED_DOUBLE_ELEMENTS`
- `PACKED_ELEMENTS` は対象外
- 分かれ目はwrite barrier

write barrier

- V8のGCは世代別で、若い世代と古い世代を別々に掃除する
- だからGCは、どのオブジェクトがどこを指しているかを知っておく必要がある
- ポインタを書くたびにGCへ知らせる仕組みがwrite barrier

memcpyでポインタを運ぶと

- `memcpy` はビット列を写すだけで、GCに知らせない
- GCは新しくできた参照に気づけない
- まだ使っているオブジェクトが回収され、use-after-freeになる

どの型なら memcpy できるか

ElementsKind	中身	ポインタ	memcpy
PACKED_SMI_ELEMENTS	小さな整数そのもの	なし	できる
PACKED_DOUBLE_ELEMENTS	生のdouble	なし	できる
PACKED_ELEMENTS	文字列やオブジェクトへの参照	あり	できない

サブ配列を結果配列へコピー

同じ長さのサブ配列を2つコピーする

PACKED_SMI_ELEMENTS



GCへの通知 **0**回

PACKED_ELEMENTS



GCへの通知 **0**回

hole埋めをやめる

- `AllocateFixedDoubleArrayWithHoles` は全スロットをholeの値で埋めてから返す
- 2パス方式なら、第2パスで全スロットが必ず埋まる
- 初期化しない `AllocateFixedArray` で確保して、`FixedDoubleArray` として使う

holeを詰める
flat特有の仕様との格闘

```
> [1, , 3].flat()
[ 1, 3 ]
> [1, , 3].map(n => n * 2)
[ 2, <1 empty item>, 6 ]
```

- `map` はholeを残す
- `flat` はholeを飛ばして詰める

最適化するうえで厄介なところ

ソースの `.length` は 5



↓ `flat()`

結果の長さは 3



- `.length` と結果の要素数が合わない
- 長さを先に出すには、holeを数える必要がある

HOLEY配列の扱い

- `.length` をそのまま結果の長さにできない
- 1個ずつ見て、hole以外を数える
- HOLEYなサブ配列は `memcpy` できず、1個ずつコピー

```
// src/builtins/array-flat.tq
let element: JSAny;
try {
  element = fastOW.LoadElementNoHole(index) otherwise FoundHole;
} label FoundHole {
  index++;
  continue;
}
```

- LoadElementNoHole はholeを見つけると FoundHole へ飛ぶ
- そのインデックスを飛ばすので、holeが詰まる
- HOLEY配列でも確保は1回だけ

holeを詰める

入力は .length が 10 の配列



第1パスで数えた長さは 0

第1パスでholeでない要素を数える

確保 0 回 書き込み 0 回 飛ばしたhole 0 個

V8独自言語Torqueでの実装

Torque	V8の組み込み関数を書く専用言語
fast path	条件がそろったときだけ通る速い経路
slow path	どんな入力でも正しく動く従来の経路
bailout	前提が崩れたときにslow pathへ逃げること

- 従来の `flat` はTorqueの `FlattenIntoArrayFast` と `FlattenIntoArraySlow`
- その手前に `TryFastFlat` を追加

```
// src/builtins/array-flat.tq
try {
  return TryFastFlat(o, len, depthSmi) otherwise SlowFastPath;
} label SlowFastPath {}

const a: JSReceiver = ArraySpeciesCreate(context, o, 0);
```

条件を満たさなければ従来のslow pathへ

slow pathに戻るとき

- `Symbol.species` が上書きされている
- ソース配列がProxy
- サブ配列にProxyがある
- 配列がfast modeでない (dictionary modeなど)
- 長さがSmiに収まらない
- ネストが深すぎる

明示的スタックによる反復処理

- 再帰で書くと、Torqueコンパイラがスタックオーバーフローで落ちた
- そこでスタックを自前で持つループにした

```
// src/builtins/array-flat.tq  
if (stack.length >= kMaxFlatFastStackEntries) goto Bailout;  
stack.Push(currentArray);  
stack.Push(nextIndex);  
stack.Push(currentDepth);
```

- 1段ごとに配列、再開位置、深さの3つを積む
- 積めるのは3072個まで、つまり深さ1024まで

明示的スタック

外側の配列から見ていく

入力は `[X, Y].flat(2)` `X = ["a", Z]` `Z = ["b", "c"]` `Y = ["d"]`

見ている配列

外側

X

Y

残りの深さ 2

スタック

空

結果

安全性の担保と Bailout

- 前提がそろっている間だけ速いコードで進む
- 前提が崩れたら、その場でslow pathへ

どこでbailoutするか

場所	確認すること
fast pathの入口	長さがSmiの範囲内か、配列がfast modeか
走査中	<code>Recheck()</code> で配列の構造が変わっていないか
第2パスの最後	書き込んだ要素数と、第1パスで数えた長さが一致するか

2パス方式のベンチマーク

- d8で、要素1,024個のサブ配列20,000個を `flat()` して計測
- 50回の中央値

配列型	パッチ適用 (median)	main (median)	改善倍率
SMI (整数)	39.32 ms	181.06 ms	~4.6x
DOUBLE (浮動小数点数)	48.21 ms	224.80 ms	~4.7x
OBJECT (文字列)	79.56 ms	190.80 ms	~2.4x

バルクコピーまで入れたベンチマーク

- d8 (arm64) で、V8 14.6.206とmainを比較
- 要素1,000個のサブ配列1,000個を `flat(2)` して計測
- 8回の最小値

サブ配列の型	最適化前	最適化後	速度比
PACKED_SMI_ELEMENTS	6.09 ms	0.26 ms	約24x
PACKED_DOUBLE_ELEMENTS	7.56 ms	0.53 ms	約14x
PACKED_ELEMENTS	5.92 ms	3.87 ms	約1.5x
HOLEY_SMI_ELEMENTS	5.91 ms	3.45 ms	約1.7x
HOLEY_DOUBLE_ELEMENTS	7.49 ms	4.66 ms	約1.6x
HOLEY_ELEMENTS	6.12 ms	3.85 ms	約1.6x

3つの実装の比較

	最適化前	2パス方式	バルクコピー
結果配列の確保	足りなくなるたびに作り直す	長さを数えて1回だけ	1回だけ
サブ配列の処理	再帰で中へ入り1個ずつ追加	1個ずつ直接書く	数値配列は <code>memcpy</code> でまとめて
要素ごとの確認	ある	ある	数値配列ではない
速度の目安	1x	約5x	約20x

V8に大きな変更を
入れるまでの流れ

まずv8-devで提案する

- 大きな変更は、いきなりパッチを出さない
- 先にレビュアーと方針をすり合わせる
- v8-devに投稿すると、誰かが反応してくれる

<https://groups.google.com/g/v8-dev/c/8ROaTLSDXkM>

**「良さそうだね
パッチ出して、議論はそっちでしょうか」**

Leszek Swirskiさんの返信

GerritにCLを出す

- GerritにCLをアップロード
- レビューアーには、変更したファイルのオーナーであるLeszekさんを入れる
- 最初の版はC++のruntime関数で書いていた

最初の指摘

- Leszekさん「このfast pathはruntime関数ではなく、Torque/CSAで実装するのがよい」
- runtimeの呼び出しコストを払わずに済む
- V8では普通、fast pathはTorqueかCSAで書き、runtimeを呼ぶのはslow pathだけ
- パッチセット2でTorqueに書き直す

マージまで

- マージには2人の `Code-Review+1` が要る
- 私にはtry jobもCQも動かす権限がないので、レビュアーに頼む

ラベル	動き
<code>Commit-Queue+1</code>	try botでテストだけ回す (dry run)
<code>Commit-Queue+2</code>	テストが通ったらそのままマージ

Chrome 147 (V8 14.7) でリリース

[array] Add Torque fast path for Array.prototype.flat

Reviewed-on: <https://chromium-review.googlesource.com/c/v8/v8/+7526287>

Reviewed-by: Olivier Flückiger <olivf@chromium.org>

Reviewed-by: Leszek Swirski <leszeks@chromium.org>

Commit-Queue: Olivier Flückiger <olivf@chromium.org>

Commit-Queue: Leszek Swirski <leszeks@chromium.org>

Cr-Commit-Position: refs/heads/main@{#105498}

ClusterFuzzとの戦い

マージの翌日、GoogleのClusterFuzzが3件発見
buganizer-systemからメールが届く

- crbug 488366773
- crbug 488586038
- crbug 489008235

原因は `GetPackedElementsKind` が

`HOLEY_DOUBLE_ELEMENTS` を `PACKED_DOUBLE_ELEMENTS` として扱っていたこと

```
// src/builtins/array-flat.tq
macro GetPackedElementsKind(kind: ElementsKind): ElementsKind {
  if (kind == ElementsKind::HOLEY_SMI_ELEMENTS)
    return ElementsKind::PACKED_SMI_ELEMENTS;
  if (kind == ElementsKind::HOLEY_DOUBLE_ELEMENTS)
    return ElementsKind::PACKED_DOUBLE_ELEMENTS;
  if (kind == ElementsKind::HOLEY_ELEMENTS)
    return ElementsKind::PACKED_ELEMENTS;
  return kind;
}

const sourceKind: ElementsKind =
  GetPackedElementsKind(source.map.elements_kind);
```

なぜクラッシュしたのか

- `HOLEY_DOUBLE_ELEMENTS` はholeを含むので、`.length` と要素数が合わない
- 第1パスで出した長さを実際の要素数がずれる
- `V8_ENABLE_UNDEFINED_DOUBLE` が有効だと、`FixedDoubleArray` にundefinedも入る
- 第2パスの `UnsafeCast<Number>` がundefinedを受け取ってクラッシュ

```
// src/builtins/array-flat.tq
doubleElements.values[targetIndex] =
  Convert<float64_or_undefined_or_hole>(UnsafeCast<Number>(element));
```

修正と Close

- `GetPackedElementsKind` を消し、`source.map.elements_kind` を直接見る
- ショートカットは本当にPACKEDなときだけ
- 回帰テストを足し、CLの `Bug:` にissue番号を書く
- マージ後、ClusterFuzzが自動で再評価してClose

マージ後も

ClusterFuzzが品質を守っている

まとめ

- 従来のflatは、結果配列を確保し直しながら1個ずつ追加していた
- 2パス方式で長さと型を先に求め、確保を1回にした
- 数値だけのサブ配列はwrite barrierが要らないので、`memcpy` でまとめてコピー
- holeは第1パスで数えて、第2パスで飛ばす
- Torqueの `TryFastFlat` で実装し、前提が崩れたらslow pathへ
- v8-devで方針を合わせてからGerritに出し、2人の `Code-Review+1` とCQでマージ
- マージ翌日にClusterFuzzが見つけたバグは、回帰テストを足して修正

参考資料

- X: x.com/bunjavascript/status/2011898197330075845
- WebKit: [WebKit/WebKit#56035](#)
- v8-dev: [\[Design/Perf\] C++ fast path for Array.prototype.flat \(2-pass, V8\)](#)
- Gerrit: [\[array\] Add Torque fast path for Array.prototype.flat](#)
- Gerrit: [\[array\] Fix flat fast path crash on HOLEY_DOUBLE with undefined](#)
- Gerrit: [\[array\] Add bulk copy shortcuts to Array.flat fast path](#)
- Zenn: [Chromium\(V8\)のArray.prototype.flatを最大約5倍高速化した](#)
- Zenn: [Chromium\(V8\)のArray.prototype.flatをバルクコピーでさらに約5倍高速化した](#)

ご清聴ありがとうございました